

CLIENT SIDE AND SERVER SIDE STATE MANAGEMENT IN PHP

State management means: *How does the website remember things about a user between requests?*

HTTP is "stateless," meaning every request is like a new conversation. If you visit a page, then click another page, the server does not automatically know it is still you. State management solves that.

1. Client-side state management (stored in the browser)

The data is stored **on the user's device**.

Example:

You visit a shopping website and add a product to your cart.

The website can store:

Cart:
- Laptop
- Mouse

Inside your browser using things like:

- Cookies
- Local Storage
- Session Storage

Example: Cookie in PHP

```
<?php
setcookie("username", "Alex", time()+3600);
?>
```

This tells the browser:

"Remember that the username is Alex for the next hour."

Later:

```
<?php
echo $_COOKIE["username"];
?>
```

Output:

Alex

The browser sends the cookie back to the server with future requests.

Real-life example

A website remembers:

- Your language preference
- Theme (dark/light mode)
- "Remember me" login option

Because that information is saved in your browser.

Advantages

Simple

Reduces server storage

Works even after closing the browser (depending on storage type)

Disadvantages

Users can view/change the data

Limited storage

Not good for sensitive information

Never store things like:

```
password  
credit card number  
private user data
```

In client-side storage.

2. Server-side state management (stored on the server)

The data is stored **on the web server**, not in the user's browser.

PHP commonly uses:

- Sessions
- Databases
- Server files
- Cache systems

PHP Sessions (most common)

A session works like this:

1. User logs in.
2. PHP creates a session on the server.
3. PHP gives the user a session ID.
4. Browser stores only that ID.
5. Server uses the ID to find the user's data.

Example:

```
<?php
session_start();

$_SESSION["username"] = "Alex";

echo "Session created";
?>
```

Another page:

```
<?php
session_start();

echo $_SESSION["username"];
?>
```

Output:

Alex

The actual value:

username = Alex

is stored on the server.

The browser only has something like:

PHPSESSID = a82jd92ks83

Real-life example

A bank website:

You log in:

Username: Alex
Password: ****

The server creates:

Session ID:
8273abc

Server memory:
8273abc → Alex is logged in

Your browser stores:8273abc

The server knows who you are.

Client-side vs Server-side comparison

Feature	Client-side	Server-side
Stored where?	User browser	Server
Examples	Cookies, Local Storage	Sessions, Database
Security	Lower	Higher
Storage size	Small	Larger
User can modify?	Yes	Usually no
Speed	Faster (no server lookup)	Requires server access
Good for	Preferences	Login, private data

Simple analogy

Imagine a library.

Client-side state

The librarian gives you a card:

```
My favorite books:
- Harry Potter
- Dune
```

You keep the card.

The librarian does not store it.

Server-side state

The librarian keeps a record:

```
Customer 12345:
- Borrowed books
- Due dates
- History
```

You only carry your ID card.

In a PHP application, you often use both

Example: Online shopping site

Client-side:

```
Cookie:  
theme = dark  
language = English
```

Server-side:

```
Session:  
user_id = 501  
cart_items = [12, 15, 20]
```

Database:

```
User:  
501  
Name: Alex  
Orders: 25
```

A common PHP architecture is:

```
Browser  
  |  
  | cookie (session ID)  
  ↓  
PHP Server  
  |  
  | session data  
  ↓  
Database
```

In short:

- **Client-side state = "The user remembers."**
- **Server-side state = "The server remembers."**